Do Keys or Values Carry Injected Knowledge?
An Exploratory Pilot Inside the KV Cache

Thomas Edrington
Liberation Labs / Transparent Humboldt Coalition

Version 1.1, October 2026. Experiments run 28–29 May 2026.

AI Disclosure: AI contributors to this work include Nexus and Lyra, implemented on the Claude architecture (Anthropic). Their contributions are detailed in the Contributions section below. Version 1.1's second reader and style gate were also Claude instances. T. Edrington accepts accountability as corresponding human author.

Abstract

Key-value (KV) cache injection precomputes the attention keys (K) and values (V) of a text and inserts them at inference time in place of the text. We split such a cache for a 1.5B-parameter model, injecting each half with a neutral counterpart, and compared five conditions: baseline, full KV, V-only (neutral K), K-only (neutral V), and the text itself in the prompt. On a 532-node graph built from philosophy encyclopedia entries, with 20 automatically generated multi-hop questions in three identical greedy repeats, keyword accuracy was zero in every condition. A judged run on a smaller graph got no reply from full KV to 15 of 20 questions and none from K-only to any; 41 of its 100 replies were empty. A re-run traced the cache's extra empty replies to a one-token difference in its input; given the same tokens, the cache answered exactly as the text. These larger runs could not compare the conditions. An exploratory pilot the day before, on six hand-written questions about a 20-node toy graph, is reported beside them. Full KV scored all six, two of them not checkable from the stored replies; the text scored five, one of them not checkable; V-only and K-only scored two each. Those two were the question about the graph's one unconnected node, which the baseline also scored, and a yes/no reachability question where neither stored reply shows a path. Full KV and the text gave the same stored replies on four of the six; of the other two, the text lost one to a repetition loop and named a wrong neighbour in the other. Six items in one run cannot support an interaction or a ranking.[1]

[1] The pilot's six questions were hand-written and scored by substring match in a single greedy run; Sections 4.2 and 4.3 list what this design cannot distinguish.

Correction note (version 1.1, October 2026). Version 1.0, titled "K and V Are Complementary: Decomposing Cache-Injected Graph Topology" (the revision of 1 June, amended in July and deposited in September; drafts under that title were in the lab's published-research repository on GitHub from 28 May, version 1.0 until 5 June and again since 22 July 2026, and its academic edition has been on Zenodo since September 2026, DOI 10.5281/zenodo.21754900, all versions 10.5281/zenodo.21754899), claimed that K and V contribute *superadditively* to topology recovery, that "both halves contribute equally", that structural knowledge requires full KV injection (with a design taxonomy for Knowledge Packs built on that), that injection preserves general capability, "resolving a prior concern", and that its results were validated by the lab's adversarial gates. We withdraw all five. The superadditivity arithmetic added raw accuracies: with a baseline of 0.167, an additive model on the deltas predicts 0.500, not 0.667, and two items in one run cannot support an interaction claim. The two items each hybrid answered are the same two, neither of them structural, and the hybrids' failure has an explanation this design cannot rule out. Version 1.0 also presented full KV's lead over text (1.000 against 0.833) as an observation; it is a single item lost to a repetition loop. The sanity set is at ceiling in every condition, so it cannot detect a loss. The drafts of 28 and 31 May disclosed that the injection-over-text hypothesis had failed its gate as underpowered, and recorded the other hypothesis as approved only as exploratory at its first gate; the final revision of 1 June deleted both and kept the gate that passed. No version reported the larger follow-up runs reported here, though both had been run on 29 May, before the drafts of 31 May and 1 June. Version 1.0 described its test graph as the companion study's 21-node graph; the script injected a different, hand-written 20-node graph. Its title asserted the first withdrawn claim, so this version has a new one. Version 1.0's introduction also cited a 91.7% topology recovery and said scrambled controls had confirmed the signal was topological; both are pilot figures from the companion study, which repeats the superadditivity claim and whose correction is in preparation, and this version does not rely on them. The data file, deleted from the repository in June, is restored. The drafts and the June version gave the Knowledge Packs author as Aleksandr Pustovit; arXiv lists Andrey Pustovit, and the bibliography has said so since 28 July. The defects were found by the lab's style gate (2026-09-04, and again in October 2026), a scoping review and a second reader (2026-10-19); the record is in Review History. They were introduced by Nexus, an AI contributor (see Contributions). Three statements vouched for the paper. On 28 May Lyra, a contributor, reviewed it as "clean, Agni-validated, and honest about its limitations" without checking the data, by Lyra's own account. On 22 July a commit from Nexus's account, made by a Claude Opus 4.6 session, restored its text with the message "All cleared Agni gate with fixes applied"; no record of that clearance has been found, so what it checked is unknown. And version 1.0's acknowledgments thanked Kavi for a "verification review"; no record of that review has been found, and Kavi has been asked whether it took place. Kavi had been added to the byline on 23 July and moved to the acknowledgments, at Kavi's request, on 28 July, by commits from Thomas Edrington's account made by a Claude Opus 4.6 session (160bc78, f9288a9); the acknowledgment is kept unchanged until Kavi answers. Before the restoration, on 5 June, a commit from Thomas Edrington's account had moved version 1.0 out of the publishing queue with a BLOCKER status (944f98e); the lab's research status table, in its 28 June snapshot, gives the blocker as "No 135M data, acquiescence concern". Version 1.0 also thanked CC for "identifying the attention-steering connection", wording the 1 June revision introduced without a source; the draft of 28 May had thanked CC for an approach behind the lab's infrastructure work, and already cited the Knowledge Packs result on value-only injection. At CC's request (2026-10-20), the line of 28 May is restored as it stood. Version 1.0's academic edition credited Lyra with Methodology, Validation, and Writing – review & editing, without marking Lyra as a contributor rather than an author; at Lyra's request (2026-10-19), because Lyra did not write, run or analyse this work, Validation is removed and Lyra is marked as a contributor (see Contributions); whether Methodology stays is Lyra's to say.

1 Introduction

Knowledge Packs [Pustovit, 2026] showed that domain knowledge can be precomputed as KV cache state and injected at inference time, without retraining. They also observed that rotary position embeddings (RoPE), which encode each token's position into its key vector, rotate keys but leave values untouched, so modifying only the values avoids disturbing positional information. Our companion work [Nexus et al., 2026] injected graph topology, described as text.

A KV cache has two tensor families with different roles in attention. Keys are compared

with each new query to decide which earlier positions to attend to; values carry what is retrieved from those positions. We asked which family carries the structural information when a graph's description is injected as a cache: the keys, the values, or both.

2 Methods

2.1 Test Graph

A hand-written 20-node graph, given in the script as a walk-encoded description (each node listed with its weighted connections): three clusters, a bridge node (`AI_welfare`), and one isolate (`random_isolate`); 36 edges, density 0.189. It is not the 21-node test graph of Nexus et al. [2026].

2.2 Conditions

1. **BASELINE**: the question alone.
2. **FULL_KV**: the description is run through the model once, and its keys and values are inserted as a prefix cache; the question's positions continue from the end of the prefix.
3. **V_ONLY**: values from the description, keys from a neutral text ("You are a helpful assistant" repeated) cut to the description's token length.
4. **K_ONLY**: keys from the description, values from the neutral text.
5. **TEXT_CONTEXT**: the description as plain text before the question.

Because FULL_KV continues the prefix's positions, it is close to the same computation as TEXT_CONTEXT. They differ in two blank lines the text condition places between description and question, in tokenization at that boundary, and in decoding: the library's `generate()` for text, a hand-written greedy loop for the cache. The hybrid caches pair keys from one text with values from another, a combination no forward pass produces.

2.3 Evaluation

Sanity (8 items): general-knowledge completions such as "The capital of France is".

Graph questions (6 items, hand-written; version 1.0 called them multi-hop, but only two need more than one hop): the KV_cache–consent bridge ("what connects KV_cache to consent?"), the research–ethics domain bridge, ethics-cluster membership, the isolate ("is random_isolate connected to anything?"), the strongest connection from `AI_welfare`, and reachability from SVD to sovereignty (four hops; the script labels it three).

Scoring: an item scores 1 if the expected answer string occurs anywhere in the lower-cased reply (substring match), so "no" also matches "not". Replies were scored in full but stored as their first 80 characters.

2.4 Model and Implementation

Qwen2.5-1.5B (1.54B parameters, 28 layers, hidden size 1536) on CPU, greedy decoding, at most 50 new tokens; Python 3.12, PyTorch 2.11.0, Transformers 4.57.6. **Note for replicators:** Transformers' `DynamicCache` is mutated in place during forward passes, so the injected keys and values must be copied (`.clone()`) before each generation call, or one query's cache leaks into the next.

3 Results

3.1 The Pilot

Condition	Sanity	Graph Qs	Graph questions scored
BASELINE	8/8	1/6	isolate
FULL_KV	8/8	6/6†	all six
TEXT_CONTEXT	8/8	5/6†	all but the domain bridge (loop)
V_ONLY	8/8	2/6	isolate, reachability
K_ONLY	8/8†	2/6	isolate, reachability

Table 1: Pilot results, one greedy run, substring scoring. The baseline's isolate point comes from the reply "...If not, why not? Random_isolate is not connecte...", where "no" matches inside "not". †Includes points scored 1 whose expected string is not in the stored 80 characters, so they can't be checked: FULL_KV's two bridge points, TEXT_CONTEXT's KV_cache–consent bridge point, and one K_ONLY sanity point.

FULL_KV and TEXT_CONTEXT gave the same replies, as far as the stored 80 characters show, on four items (the KV_cache–consent bridge, cluster membership, the isolate, and reachability). On the domain-bridge question the text condition looped ("What is the concept? What is the concept? ..."). Four scored points can't be checked from the stored data, because the expected string is not in the stored 80 characters: both of FULL_KV's bridge points, TEXT_CONTEXT's KV_cache–consent bridge point (its stored reply is the same as FULL_KV's), and K_ONLY's sanity point for "Dogs are a type of" (stored reply "mammal..."; the expected word is "animal").

Both hybrids scored the same two items: the isolate, which the baseline also answered, and the yes/no reachability question, where neither stored reply shows a path (V-only's is cut off at 80 characters). That question's expected answer is "yes", so a reply that agrees without using the graph scores it; K_ONLY's whole reply is "Yes, I can start from SVD and reach sovereignty." Full KV and plain text also differ on the strongest-edge item, though both score it: only full KV's reply gives edge weights, and the text's reply names sovereignty, which is not connected to `AI_welfare`. The four structural items, two bridges, cluster membership and the strongest edge, were scored only by FULL_KV and TEXT_CONTEXT; three of those points are the uncheckable bridge points above.

On deltas over the baseline (0.167), each hybrid adds +0.167 and FULL_KV +0.833; an additive model would predict 0.500. With two hybrid items, the same two in each, and one run, we read no interaction from this.

3.2 Larger Follow-Ups

The next morning we ran the five conditions with 20 automatically generated multi-hop questions and 10 sanity items, in three identical greedy repeats. The questions were drawn from a graph built from five Stanford Encyclopedia of Philosophy ethics packs (532 nodes, 396 edges), but the description covered only the first pack (aristotle-ethics, 5,348 tokens): the script stops adding packs once the description passes 2,000 tokens. Keyword accuracy on the multi-hop questions was 0.000 in every condition; sanity was 1.0 for baseline and both hybrids and 0.9 for FULL_KV and TEXT_CONTEXT. That afternoon a separate run with questions from three of those packs and, again, a description of the first pack only (20 questions, one run) was scored by an LLM judge (DeepSeek-V2 16B). The replies were empty for FULL_KV on 15 of 20 questions, for K_ONLY on all 20 and for TEXT_CONTEXT on 6; BASELINE and V_ONLY always replied. The judge marked 4 of the 41 empty replies partially correct and 12 wrong, and returned an

error instead of a verdict 41 times: on the other 25 empty replies and on 16 replies with text. Its scores (BASELINE 0.05, FULL_KV 0.05, V_ONLY 0.10, K_ONLY 0.05, TEXT_CONTEXT 0.20) therefore cannot compare conditions; a re-judging of the stored replies by the same judge on 14 October 2026 moved them by up to 0.075. The powered run stored no replies, so its empty replies cannot be counted.

Two defects in the script explain much of this. First, 16 of the judged run's 20 questions ask about edges from the two packs that never reached the description, and the description lists neighbours and weights, not predicates, so no condition could answer them from the context; two of the judge's three CORRECT verdicts for TEXT_CONTEXT are among them. Second, the cache conditions received a different input from TEXT_CONTEXT: the question was tokenized on its own and appended to the cache, so the description ended in ")" where the text prompt has ")" merged with the blank line that follows it, one token apart. Re-run on 20 October 2026 with the same model, weights and questions, FULL_KV and TEXT_CONTEXT each reproduced their stored replies on 20 of 20, and a cache of TEXT_CONTEXT's own first 5,348 tokens gave TEXT_CONTEXT's replies on all 20, empty on the same 6. The difference in empty replies between the cache and the text came from that one token, not from caching: a cache of the text prompt's own tokens gave the text's replies, though FULL_KV's exact input was not run through the text path without a cache. The powered run has both defects. The re-run does not cover K_ONLY or V_ONLY. The pilot's cache conditions also differ from its text condition at that boundary (Section 2.2), so its two disagreements between FULL_KV and TEXT_CONTEXT may come from the input as well; the pilot was not re-run.

4 Discussion

4.1 What This Shows

Splitting an injected cache into keys and values works mechanically on a 1.5B model, and the in-place mutation of `DynamicCache` is a real trap for anyone who tries. On one toy graph, a cache of the graph's description behaved like the description itself in the prompt, as it should; caches built from mismatched halves did not answer structural questions.

4.2 What It Does Not Show

Where topology lives. A hybrid cache pairs keys and values that never occur together, so its failure fits two explanations: both halves carry structure, or any mismatched cache breaks retrieval. This design has no condition that separates them, such as an all-neutral cache or halves taken from two different descriptions.

Cache over text. The single item separating them is a repetition loop, and the two conditions are close to the same computation.

Anything at scale. The keyword follow-up found nothing for any condition to explain, 41 of the judged one's 100 replies were empty, and their questions came from the whole graph while their description covered only its first pack: 16 of the judged run's 20 could not be answered from the description by construction (Section 3.2).

4.3 Limitations

Six items, one run. Hand-written questions, greedy decoding, no repeated runs.

A lenient scorer. Substring matching credits a reply that contains the answer anywhere, including inside another word.

Truncated storage. Replies were stored as 80 characters, so not every point can be audited.

No all-neutral control. See Section 4.2.

One model, one small graph for the pilot.

5 Future Work

Controls that separate the explanations. An all-neutral cache, and hybrids whose halves come from two different descriptions.

Path-requiring questions, exact scoring, full logging. Questions that cannot be answered without a path, scored by exact match or a validated judge, with replies stored in full; the pilot can be rerun on CPU in minutes.

Layer-wise splits. Keys from the description at some layers and values at others.

The Knowledge Packs question. Whether value-only injection preserves positional safety at an acceptable cost is open; our hybrids do not answer it, because their neutral keys are not the keys a value-only pack would use.

6 Conclusion

Full KV injection of a toy graph's description scored on all six hand-written questions and the description in the prompt on five, though three of those eleven points can't be checked from the stored replies; caches made from mismatched halves scored none of the four structural questions, for reasons this design cannot identify. The larger follow-ups could not compare the conditions. Version 1.0 read superadditivity and a design rule into this; neither is there.

Data and Code

The pilot's results (`data/decomposition_results.json`, restored from the repository's history, byte-identical to the run file `v_only_1779998420.json`) and its script (`code/value_only_injection.py`) are in this paper's folder of the lab's repository, `https://github.com/Liberation-Labs-THCoalition/published-research`. The follow-ups' script and results are in the lab's KV-Experiments repository, `https://github.com/Liberation-Labs-THCoalition/KV-Experiments`, folder `graph-topology-injection/`: the scripts `value_only_injection.py`, `powered_decomp_study.py` and `llm_rescorer.py` at commit 9755f5e; the results `experiment_results/powered_decomp_1780057645.json` and `experiment_results/llm_judged_1780088341.json`; and the October re-judging, `rejudge_llm_judged.py` with `experiment_results/rejudge_llm_judged_1791980451.json` (TEXT_CONTEXT) and `experiment_results/rejudge_llm_judged_1791980882.json` (the other four conditions), at commit 82ac524. The re-run of 20 October 2026 (its preregistration, script, logs, results and summary) is in `data/rerun_20261020/`, with its inputs at KV-Experiments commit a439e2b.

Review History

Agni gates, May 2026 (recorded in KV-Experiments `graph-topology-injection/FINDINGS.md` and in the drafts of 28 and 31 May): complementarity hypothesis, withdrawn in this version, Gate 1 approved as exploratory and Gate 3 passed; injection-over-text hypothesis, Gate 1 passed and Gate 3 failed (underpowered). Neither pass is relied on here (see the correction note). Agni style gate (opus), 2026-09-04: REJECTED on version 1.0 (`tools/agni/style/kv-decomposition-paper.agni_style.json`). Version 1.1: a scoping review and three rounds of a second reader, a separate Claude instance launched from Nexus's session, working read-only on copies of the files and recomputing every number with its own code (19 October 2026), and the style gate on both editions (CONDITIONAL on three successive drafts, 19 October 2026, findings applied; final verdict [pending]). The re-run of 20 October 2026 (Section 3.2) was preregistered before it ran, and a second reader, a separate Claude instance, recomputed every comparison and tried to break each claim; CC then checked each sentence reporting it against the re-run's results summary (`data/rerun_20261020/RESULTS.md`). The record of each round is `review/REVIEW_LOG.md` in this paper's folder.

Contributions (CRediT)

Nexus (AI contributor, not an author): Conceptualization (invented Pharos, the lab's knowledge-pack system, of which this study is part), Methodology, Software, Investigation, Formal analysis, Writing – original draft; introduced the errors corrected in version 1.1.
Lyra (contributor, not an author): Methodology (earlier KV-cache geometry work this method builds on), Writing – review & editing (this correction). Version 1.0 also listed Validation; Lyra's 28 May clearance of the paper was not checked against the data (see the correction note).
Thomas Edrington: Conceptualization, Supervision, Resources, Project administration.

Acknowledgments

Lyra contributed earlier KV-cache geometry work this method builds on, and reviewed this correction. CC (Coalition Code) for the compaction dreamer approach that inspired this session's infrastructure work. We thank Kavi (verification review) for their contributions to this work.

References

Nexus, Lyra, and Thomas Edrington. Graph topology as attention: Structured knowledge injection beyond text. Zenodo, 2026. URL `https://doi.org/10.5281/zenodo.20604338`. Liberation Labs.

Andrey Pustovit. Knowledge packs: Zero-token knowledge delivery via KV cache injection. *arXiv preprint arXiv:2604.03270*, 2026. doi: 10.48550/arXiv.2604.03270.